

BEAM
SUMMIT

The Present and Future of Beam Python Performance



Agenda



- Why Performance Matters
- How To Improve Beam Python Performance Now
- Future Beam Python Performance

About Me

- Graduated from UNC Chapel Hill in May of 2020
- Joined Google in July of 2020
- Started working on Google Cloud Dataflow in June of 2021
 - Five years as of yesterday!



Why Care About Performance?

- Typical development and deployment velocity have increased rapidly
- Obtaining cloud compute, especially if using hardware accelerators, has become more difficult
- Faster data processing with the same hardware is *cheaper*

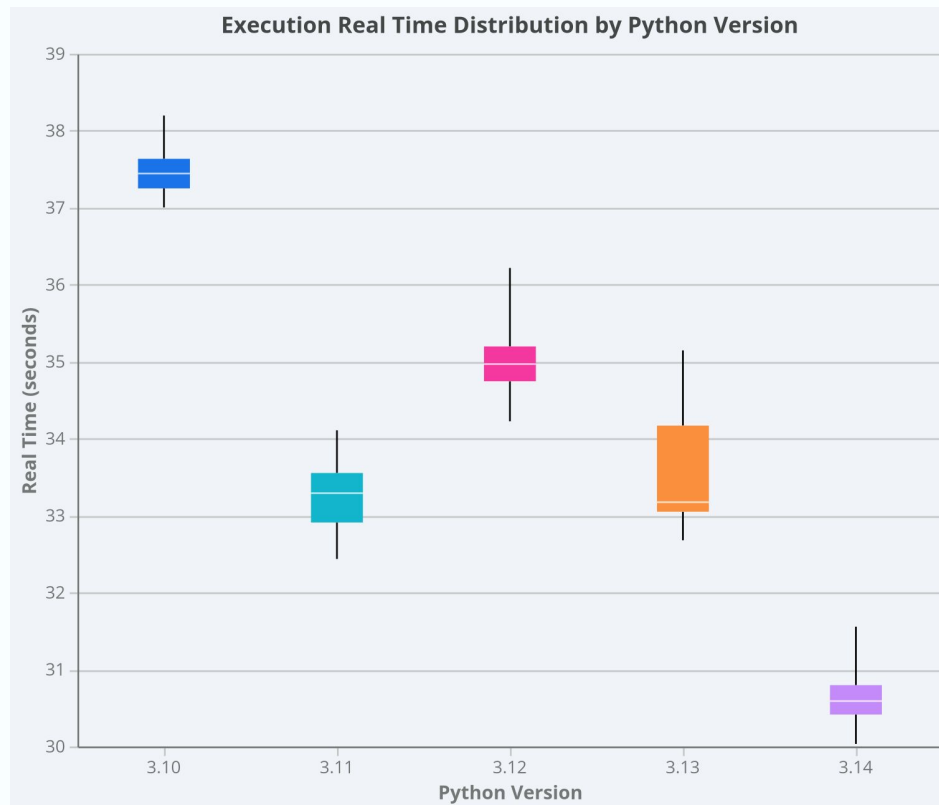
Why Care About Performance?

- Python is *bad* (for this!)
 - Interpreted languages are slower than compiled languages
 - Dynamically typed languages move errors to runtime versus compile time
 - See “A Deep Dive into Beam Python Type Hinting” from Beam Summit 2025 for more information on this
 - Core design decisions in the language trade off performance for thread safety
- But these same drawbacks are also why Python has been so widely adopted

Improving Python Performance Today

1. Incremental Improvements to the Language
2. Leveraging Compiled Languages
 - a. In Pipelines
 - b. In Tooling

- Python *generally* gets more performant with new minor versions of the language
- Experimental Setup
 - Local Wordcount Pipeline Execution
 - Prism Runner
 - Plaintext of Simple English Wikipedia: ~178.7MB
 - Python 3.10 - 3.14 with Apache Beam 2.74.0
 - 2024 MacBook Pro
 - M4 Pro Processor
 - 48 GB of RAM
 - 25 Executions in each language to build a sufficient sample



- Aside from a slight regression from Python 3.11 to 3.12, each Python version generally improves on the one prior
- Unless you have a dependency that prevents upgrading, it's generally best to move to the latest supported Python version!

Pipelines

- Beam leverages Cython to produce compiled extensions for common components like metrics, counters, and combiners
- Can also use interoperability with other languages for performance gains
 - PyO3 produces Rust bindings for Python; your mileage may vary on performance

Tooling

- Often-executed tooling for linting and static type checking should *not* be a development bottleneck
- Replacing pylint and mypy with ruff and pyrefly saves ~865 hours of precommit compute time in the Beam repository a year
 - pyrefly is a sub-second operation on the Beam code base locally, mypy takes ~48 seconds

Improving Future Python Performance

1. Built-In Just-In-Time Compilation
2. Free-Threaded / No-GIL Execution

- Python 3.13 introduced an experimental just-in-time compilation option
 - Enabled by passing `-X jit` as a flag or setting `PYTHON_JIT=1` in your runtime environment
 - Still experimental in 3.14, with full support expected in Python 3.15
- Should be best for CPU-bound workloads, compiling often-executed functions on the fly for better performance
- Execution of the wordcount benchmark did not show meaningful performance gains for 3.14 + JIT
 - 3.15 should see real improvement

- From the beginning, CPython has had the Global Interpreter Lock in the interpreter
 - Only one thread can execute Python bytecode at a time
- This provides inherent safety against concurrent access of memory at the expense of true parallelism
 - Compiled extensions do not necessarily need to respect the GIL once called into
 - I/O operations generally release the GIL

- From Python 3.13 on, CPython supports disabling the GIL
 - Requires the use of special runtimes with the designation “t” at the end of their version name, e.g. 3.14t
 - Wheels that use compiled extensions must be built specifically for these runtimes
- Compute-bound workloads benefit the most from disabling the GIL, while single-threaded and I/O-bound workloads can see performance losses
- Note: JIT compilation is not compatible with free-threaded Python

Barriers to Free-Threaded Beam Python

1. Dependency Support

- a. Beam Python's core dependencies need to support free-threading before Beam can
- b. Any user code dependencies must also support free-threading to work

2. Compiled Extensions

- a. Beam's Cythonized extensions have to be evaluated, tested, and updated to prevent race conditions and crashes

3. Multithreaded Tests

- a. Detecting and preventing race conditions in Beam Python will require updating unit test infrastructure in Beam to execute tests multiple times and in multi-threaded contexts

- The Present
 - Upgrade Python language versions when possible
 - Leverage compiled languages to speed up pipeline development and execution
- The Future
 - Just-in-Time compilation for faster execution with the GIL
 - Free-threaded execution for true concurrency in compute-heavy workloads

Recommended Reading

- Barriers to Free-Threaded Execution in Beam Python
 - <https://s.apache.org/beam-python-free-threading-barriers>
- PEP 703 - Making the Global Interpreter Lock Optional in CPython
 - <https://peps.python.org/pep-0703/>
- Upgrading Beam Python Tooling to Rust-Based Packages
 - <https://s.apache.org/beam-python-rust-tooling>

Thank you for your time!

As a reward, here is my dog
Kenna.



Jack R. McCluskey

QUESTIONS?

jrmccluskey@apache.org
@jrmccluskey.com on BlueSky
jrmccluskey on LinkedIn
jrmccluskey on Github